



East West University

CSE103: Structured Programming

Summer 2026

Project Report

Academic Portal System

Submitted to:

Md. Ashraful Haider Chowdhury

Lecturer

Department of Computer Science and Engineering

Submitted by :

Student ID	Student Name
2026-2-60-169	Md. Rafsan Janee
2026-2-60-214	Maliha Farzana
2026-2-60-220	Tanvir Mahdin Haque

Group ID : 1

Section : 10

Date Of Submission: 19-08-2026

Table of Contents

Name of content	Page number
Abstract	03
Introduction	03
Project Aim and Objectives	03
Flowchart	04
Program Output Screenshot	05
Source Code	07
Limitation of Program	24
Extra Features	25
Conclusion	25

Abstract

The Academic Portal System is a console-based, role driven application developed in C for managing academic data. It supports faculty and student interactions through structured file handling, ensuring basic data integrity and efficient record operations. The system applies modular design and abstraction principles to simulate a simplified academic environment.

Introduction

Efficient handling of academic information is critical in institutional systems. Traditional manual approaches suffer from redundancy, inconsistency, and low scalability. The Academic Portal System introduces a structured, menu-driven solution that separates functionality into Faculty and Student portals. Faculty users can manage records, attendance, and assessments, while students can access personal data and utility tools. The system emphasizes modular programming, abstraction layers, and optimized control flow to improve usability and maintainability.

Project Aims And Objectives

The primary aim of this project is to develop a role based academic portal system using C programming that supports both faculty and student functionalities.

Objectives:

- To design a dual portal system for faculty and students.
- To implement role-based authentication using faculty and student databases.
- **To enable faculty to:**
 - Add student records
 - Update student records
 - Delete student records
 - Display all records

- Search and display specific records
- Manage attendance and marks
- Create and manage quizzes
- **To enable students to:**
 - View their academic records
 - Access utility features such as stopwatch, SGPA calculator, and goal tracker
 - Participate in quizzes
- To ensure persistent data storage using file-based databases

Flowchart

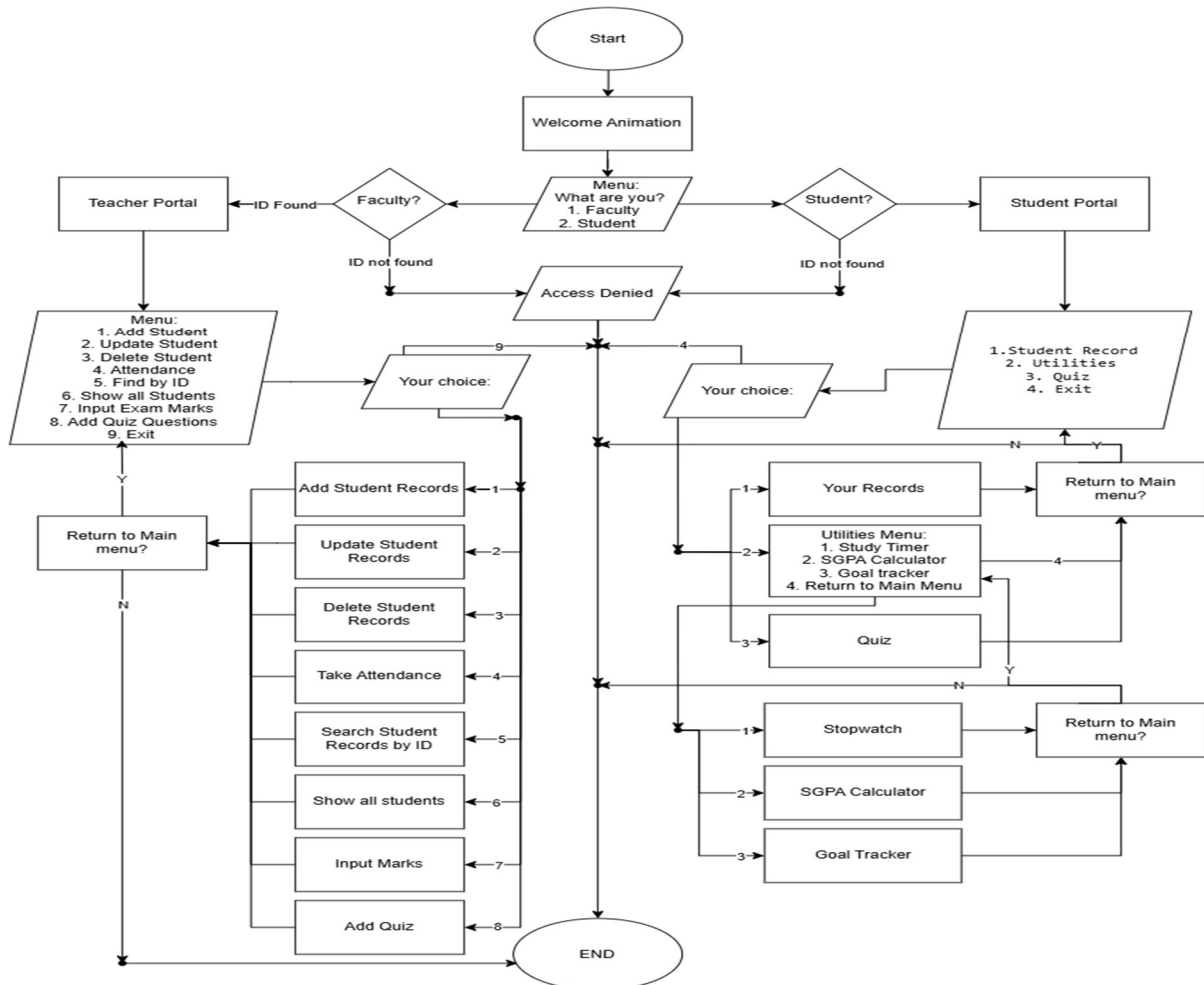


Figure 1

Program Output Screenshot

```
What are you?  
1. Faculty  
2. Student  
Enter your role: █
```

Figure 2

```
What would you like to do?  
1. Add student  
2. Update Student Record  
3. Delete Student Record  
4. Attendance  
5. Find by ID  
6. Show all Students  
7. Input Exam Marks  
8. Add Quiz Questions  
9. Exit  
Enter your choice: 6  
Sort by?  
1. Id  
2. Mark  
Enter choice: 1
```

Figure 3

```
Your Records:  
ID: 169  
Name: Rafsan  
Dept: CSE  
Semester: 3  
Grade: A+  
Marks: 81.00  
Attendance: 21 (100.00 %)  
CGPA: 3.92  
Return to Main Menu? (Y/N)
```

Figure 4

```

All student records:
ID: 030 | Name: Ajmain | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 10.00 | Grade: F | CGPA: 0.00
ID: 043 | Name: Abir | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 20.00 | Grade: F | CGPA: 0.00
ID: 045 | Name: Salman | Dept: ICE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 30.00 | Grade: F | CGPA: 0.00
ID: 049 | Name: Effat | Dept: CSE | Semester: 2 | Attendance: 16 ( 76.19 %) | Marks: 40.00 | Grade: D | CGPA: 0.00
ID: 072 | Name: Mahi | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 50.00 | Grade: C+ | CGPA: 0.00
ID: 079 | Name: Turjyo | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 60.00 | Grade: B | CGPA: 0.00
ID: 092 | Name: Gourav | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 70.00 | Grade: A- | CGPA: 0.00
ID: 111 | Name: Zara | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 80.00 | Grade: A+ | CGPA: 0.00
ID: 160 | Name: Anik | Dept: CSE | Semester: 1 | Attendance: 19 ( 90.48 %) | Marks: 90.00 | Grade: A+ | CGPA: 0.00
ID: 162 | Name: Hemal | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 80.00 | Grade: A+ | CGPA: 0.00
ID: 165 | Name: Prachurjya | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 70.00 | Grade: A- | CGPA: 0.00
ID: 167 | Name: Momtahina | Dept: CSE | Semester: 1 | Attendance: 21 ( 100.00 %) | Marks: 70.00 | Grade: A- | CGPA: 0.00
ID: 169 | Name: Rafsan | Dept: CSE | Semester: 3 | Attendance: 21 ( 100.00 %) | Marks: 81.00 | Grade: A+ | CGPA: 3.92
ID: 170 | Name: Tahira | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 90.00 | Grade: A+ | CGPA: 0.00
ID: 188 | Name: Bilash | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 80.00 | Grade: A+ | CGPA: 0.00
ID: 192 | Name: Ekram | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 70.00 | Grade: A- | CGPA: 0.00
ID: 195 | Name: Mithila | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 60.00 | Grade: B | CGPA: 0.00
ID: 204 | Name: Badhon | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 50.00 | Grade: C+ | CGPA: 0.00
ID: 205 | Name: Shohan | Dept: CSE | Semester: 1 | Attendance: 20 ( 95.24 %) | Marks: 60.00 | Grade: B | CGPA: 0.00
ID: 214 | Name: Maliha | Dept: CSE | Semester: 1 | Attendance: 20 ( 95.24 %) | Marks: 70.00 | Grade: A- | CGPA: 0.00
ID: 218 | Name: Imtiyaz | Dept: CSE | Semester: 1 | Attendance: 21 ( 100.00 %) | Marks: 80.00 | Grade: A+ | CGPA: 0.00
ID: 220 | Name: Tanvir | Dept: CSE | Semester: 1 | Attendance: 20 ( 95.24 %) | Marks: 90.00 | Grade: A+ | CGPA: 0.00
ID: 300 | Name: Tasin | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 80.00 | Grade: A+ | CGPA: 0.00
ID: 319 | Name: Billal | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 70.00 | Grade: A- | CGPA: 0.00
ID: 324 | Name: Arman | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 60.00 | Grade: B | CGPA: 0.00
ID: 325 | Name: Akayed | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 6.00 | Grade: F | CGPA: 0.00
ID: 333 | Name: Sadika | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 50.00 | Grade: C+ | CGPA: 0.00
ID: 347 | Name: Ridit | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 50.00 | Grade: C+ | CGPA: 0.00
ID: 463 | Name: Ghose | Dept: CSE | Semester: 1 | Attendance: 16 ( 76.19 %) | Marks: 60.00 | Grade: B | CGPA: 0.00
ID: 563 | Name: Roza | Dept: CSE | Semester: 1 | Attendance: 15 ( 71.43 %) | Marks: 70.00 | Grade: A- | CGPA: 0.00
Return to Main Menu? (Y/N)

```

Figure 5

```

Enter your target CGPA: 3.95

Current CGPA: 3.92
Target CGPA: 3.95
You need 0.03 more to reach your goal.
Return to Main Menu? (Y/N)

```

Figure 6

Source Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
#include <windows.h>

#define MAX_STUDENTS 100

typedef struct {
    int id, semester, attendance;
    float marks, cgpa;
    char name[50], dept[20], grade[3];
} Student;

typedef struct {
    char question[200];
    char optA[100], optB[100], optC[100], optD[100];
    char correct;
} Quiz;

int loadStudents(Student s[]); void saveStudents(Student s[], int n); int
getTotalClasses(void);
void sortStudents(Student s[], int n, int byMarks); void printStudent(Student *s,
int totalClass);
int findStudent(Student s[], int n, int id); void assignGrade(Student *s); void
headerf(); void pressEnter();

void teacherPortal(void); void stdPortal(int userid);

void addStd(void); void stdrecords(int userid);
void searchStd(void); void stopwatch(void);
void updateStd(void); void quiz(int userid);
void deleteStd(void); void addQuiz(void);
void showStds(void); void utilities(int userid); void goalTracker(int
userid);
void attendance(void); void sgpa(int userid);
void marksInput(void); void CGPA(int userid, float newSGPA);

void headerf() {
```

```

        printf("=====\n");
    );
    printf("||                               ||\n");
    );
    printf("||                               Academic System
Portal                               ||\n");
    printf("||                               ||\n");
    );
    printf("=====\n");
    );
}

void pressEnter() {
    char junk;

    printf("Press Enter to Continue...\n");

    while ((junk = getchar()) != '\n' && junk != EOF);
    getchar();
}

int loadStudents(Student s[]) {
    FILE *fp = fopen("stdDB.txt", "r");
    int n = 0;

    if (fp == NULL) {
        return 0;
    }
    while (n < MAX_STUDENTS &&
        fscanf(fp, "%d %49s %19s %d %d %f %2s %f",
            &s[n].id, s[n].name, s[n].dept, &s[n].semester,
            &s[n].attendance, &s[n].marks, s[n].grade, &s[n].cgpa) == 8) {
        n++;
    }

    fclose(fp);
    return n;
}

void saveStudents(Student s[], int n) {
    FILE *fp = fopen("stdDB.txt", "w");

    if (fp == NULL) {
        printf("Error writing student records\n");
    }
}

```



```

        return;
    }

    for (int i = 0; i < n; i++) {
        fprintf(fp, "%d %s %s %d %d %.2f %s %.2f\n",
                s[i].id, s[i].name, s[i].dept, s[i].semester,
                s[i].attendance, s[i].marks, s[i].grade, s[i].cgpa);
    }

    fclose(fp);
}

int getTotalClasses(void) {
    FILE *fp = fopen("classCount.txt", "r");
    int totalClass = 1;

    if (fp != NULL) {
        fscanf(fp, "%d", &totalClass);
        fclose(fp);
    }

    return totalClass;
}

void sortStudents(Student s[], int n, int byMarks) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            int swap = byMarks ? (s[j].marks < s[j + 1].marks) : (s[j].id > s[j + 1].id);

            if (swap) {
                Student temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }
}

void printStudent(Student *s, int totalClass) {
    float attendancePercent = (s->attendance * 100.0f) / totalClass;

    printf("ID: %03d\n"
           "Name: %s\n"
           "Dept: %s\n"

```

```

        "Semester: %d\n"
        "Attendance: %d (%.2f %%)\n"
        "Marks: %.2f\n"
        "Grade: %s\n"
        "CGPA: %.2f\n",
        s->id, s->name, s->dept, s->semester,
        s->attendance, attendancePercent, s->marks,
        s->grade, s->cgpa);
    }

int findStudent(Student s[], int n, int id) {
    for (int i = 0; i < n; i++) {
        if (s[i].id == id)
            return i;
    }

    return -1;
}

void assignGrade(Student *s) {
    if (s->marks >= 80 && s->marks <= 100)        strcpy(s->grade, "A+");
    else if (s->marks >= 75)    strcpy(s->grade, "A");
    else if (s->marks >= 70)    strcpy(s->grade, "A-");
    else if (s->marks >= 65)    strcpy(s->grade, "B+");
    else if (s->marks >= 60)    strcpy(s->grade, "B");
    else if (s->marks >= 55)    strcpy(s->grade, "B-");
    else if (s->marks >= 50)    strcpy(s->grade, "C+");
    else if (s->marks >= 45)    strcpy(s->grade, "C");
    else if (s->marks >= 40)    strcpy(s->grade, "D");
    else                        strcpy(s->grade, "F");
}

int main(void) {
    headerf();
    Sleep(1200);

    int userIn;

    do {
        printf("What are you?\n1. Faculty\n2. Student\nEnter your role: ");
        scanf("%d", &userIn);

        if (userIn != 1 && userIn != 2) {
            system("cls");

```

```

        printf("Invalid Input\n");
        Sleep(500);
    }
} while (userIn != 1 && userIn != 2);

if (userIn == 1) {
    int userId, fileId;
    char faculty[50], dept[20];
    int found = 0;

    system("cls");
    printf("Welcome to the Faculty Portal\nEnter your ID: ");
    scanf("%d", &userId);

    FILE *fp = fopen("facultyDB.txt", "r");

    if (fp == NULL) {
        printf("File not found\n");
        return 0;
    }

    while (fscanf(fp, "%d %49s %19s", &fileId, faculty, dept) == 3) {
        if (fileId == userId) {
            found = 1;
            break;
        }
    }

    fclose(fp);

    if (!found) {
        printf("Access denied\n");
        return 0;
    }

    system("cls");
    printf("Welcome %s (%s Department)\n", faculty, dept);
    Sleep(1000);
    teacherPortal();
} else {
    int userid;

    system("cls");
    printf("Welcome to the Student Portal.\nWhat is your ID?: ");

```

```

        scanf("%d", &userid);

        stdPortal(userid);
    }

    return 0;
}

void teacherPortal(void) {
    int choice;

    do {
        system("cls");
        headerf();

        printf("What would you like to do?\n1. Add student\n");
        printf("2. Update Student Record\n3. Delete Student Record\n");
        printf("4. Attendance\n5. Find by ID\n6. Show all Students\n");
        printf("7. Input Exam Marks\n8. Add Quiz Questions\n9. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice < 1 || choice > 9) {
            printf("\nInvalid Choice!\n");
            Sleep(600);
            continue;
        }
        switch (choice) {
            case 1: addStd(); break;
            case 2: updateStd(); break;
            case 3: deleteStd(); break;
            case 4: attendance(); break;
            case 5: searchStd(); break;
            case 6: showStds(); break;
            case 7: marksInput(); break;
            case 8: addQuiz(); break;
            case 9: return;
        }
    } while (1);
}

void addStd(void) {
    FILE *fp = fopen("stdDB.txt", "a");
    Student s;

```

```

    if (fp == NULL) {
        printf("Failed to open file\n");
        return;
    }

    printf("Enter: ID Name Dept Semester Attendance Marks Grade CGPA:\n");
    scanf("%d %49s %19s %d %d %f %2s %f",
        &s.id, s.name, s.dept, &s.semester,
        &s.attendance, &s.marks, s.grade, &s.cgpa);

    assignGrade(&s);

    fprintf(fp, "%d %s %s %d %d %.2f %s %.2f\n",
        s.id, s.name, s.dept, s.semester,
        s.attendance, s.marks, s.grade, s.cgpa);

    fclose(fp);

    printf("Student added successfully\n");
    pressEnter();
}

void searchStd(void) {
    Student s[MAX_STUDENTS];
    int searchId, totalClass;
    int n = loadStudents(s);

    if (n == 0) {
        printf("No records found\n");
        return;
    }

    printf("Enter ID to search: ");
    scanf("%d", &searchId);

    int index = findStudent(s, n, searchId);

    system("cls");

    if (index == -1) {
        printf("Student not found\n");
    } else {
        totalClass = getTotalClasses();
    }
}

```

```

        printf("Student Found:\n");
        printStudent(&s[index], totalClass);
    }
    pressEnter();
}

void updateStd(void) {
    Student s[MAX_STUDENTS];
    int searchId;
    int n = loadStudents(s);

    if (n == 0) {
        printf("No records found\n");
        return;
    }

    printf("Enter ID to update: ");
    scanf("%d", &searchId);

    int i = findStudent(s, n, searchId);

    if (i == -1) {
        printf("Student was not found\n");
    } else {
        printf("Enter NEW data:\n");
        printf("ID Name Dept Semester Attendance Marks Grade CGPA:\n");

        scanf("%d %49s %19s %d %d %f %2s %f",
            &s[i].id, s[i].name, s[i].dept,
            &s[i].semester, &s[i].attendance,
            &s[i].marks, s[i].grade, &s[i].cgpa);

        assignGrade(&s[i]);
        saveStudents(s, n);

        printf("Student updated successfully\n");
    }
    Sleep(1000);
}

void deleteStd(void) {
    Student s[MAX_STUDENTS];
    int searchId;
    int n = loadStudents(s);

```

```

    if (n == 0) {
        printf("No records found\n");
        return;
    }

    printf("Enter ID to delete: ");
    scanf("%d", &searchId);

    int index = findStudent(s, n, searchId);

    if (index == -1) {
        printf("Student was not found\n");
    } else {
        for (int i = index; i < n - 1; i++)
            s[i] = s[i + 1];

        saveStudents(s, n - 1);
        printf("Student record deleted successfully\n");
    }
    Sleep(1000);
}

void showStds(void) {
    Student s[MAX_STUDENTS];
    int choice;
    int n = loadStudents(s);

    if (n == 0) {
        printf("No records found\n");
        return;
    }

    do {
        printf("Sort by?\n1. ID\n2. Mark\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
    } while (choice != 1 && choice != 2);

    for (int i = 0; i < n; i++) {
        assignGrade(&s[i]);
    }
    sortStudents(s, n, choice == 2);
}

```

```

    int totalClass = getTotalClasses();

    system("cls");
    printf("All student records:\n");
    for (int i = 0; i < n; i++) {
        float attendancePercent = (s[i].attendance * 100.0f) / totalClass;

        printf("ID: %03d | Name: %-10s | Dept: %-3s | Semester: %-1d | "
               "Attendance: %-3d (%8.2f %%) | Marks: %-7.2f | "
               "Grade: %-4s | CGPA: %-3.2f\n",
               s[i].id, s[i].name, s[i].dept, s[i].semester,
               s[i].attendance, attendancePercent, s[i].marks,
               s[i].grade, s[i].cgpa);
    }
    pressEnter();
}

void attendance(void) {
    Student s[MAX_STUDENTS];
    int n = loadStudents(s);

    if (n == 0) {
        printf("No records found\n");
        return;
    }

    int totalClass = getTotalClasses() + 1;
    char status;

    printf("Taking attendance:\n");

    for (int i = 0; i < n; i++) {
        printf("%d %s (P/A): ", s[i].id, s[i].name);
        scanf(" %c", &status);

        if (status == 'P' || status == 'p')
            s[i].attendance++;
    }

    saveStudents(s, n);

    FILE *fp = fopen("classCount.txt", "w");
    if (fp != NULL) {
        fprintf(fp, "%d", totalClass);
    }
}

```



```

        fclose(fp);
    }

    printf("Attendance updated successfully\n");
    printf("\nTotal Classes Taken: %d\n\n", totalClass);
    pressEnter();
}

void marksInput(void) {
    Student s[MAX_STUDENTS];
    int n = loadStudents(s);
    char choice;

    if (n == 0) {
        printf("No records found\n");
        return;
    }

    printf("Do you want to reset marks? (Y/N)\n");
    scanf(" %c", &choice);

    if (choice == 'Y' || choice == 'y') {
        printf("WARNING: This will reset ALL marks.\nConfirm? (Y/N): ");
        scanf(" %c", &choice);

        if (choice == 'Y' || choice == 'y') {
            for (int i = 0; i < n; i++) {
                s[i].marks = 0;
                assignGrade(&s[i]);
            }
        }
    } else if (choice != 'Y' && choice != 'y') {
        float examMark;

        printf("Input Marks:\n");

        for (int i = 0; i < n; i++) {
            printf("%d %s Marks: %.2f + ", s[i].id, s[i].name, s[i].marks);
            scanf("%f", &examMark);

            s[i].marks += examMark;
            assignGrade(&s[i]);
        }
    }
}

```

```

        saveStudents(s, n);
        pressEnter();
    }

void stdPortal(int userid) {
    Student s[MAX_STUDENTS];
    int n = loadStudents(s);
    int index = findStudent(s, n, userid);

    if (index == -1) {
        printf("Student not found\n");
        return;
    }
    int choice;

    do {
        system("cls");
        headerf();
        printf("Welcome %s from %s\n", s[index].name, s[index].dept);
        printf("\nMain Menu:\n1. Your Records\n2. Utilities\n3. Quiz\n4.
Exit\n");
        printf("Your Choice: ");
        scanf("%d", &choice);

        if (choice < 1 || choice > 4)
            printf("Invalid Choice!\n");

        switch (choice) {
            case 1: stdrecords(userid); break;
            case 2: utilities(userid); break;
            case 3: quiz(userid); break;
            case 4: return;
        }
    } while (1);
}

void stdrecords(int userid) {
    Student s[MAX_STUDENTS];
    int n = loadStudents(s);
    int index = findStudent(s, n, userid);

    if (index == -1) {
        printf("Records not found!\n");
    }
}

```

```

        return;
    }

    int totalClass = getTotalClasses();

    system("cls");
    printf("Your Records:\n");
    printStudent(&s[index], totalClass);
    pressEnter();
}

void utilities(int userid) {
    int choice;

    do {
        system("cls");

        printf("What would you like to do?\n");
        printf("1. Study Timer\n2. SGPA Calculator\n3. Goal Tracker\n4. Return to
Main Menu\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice < 1 || choice > 4) {
            printf("Invalid Choice!\n");
            continue;
        }

        switch (choice) {
            case 1: stopwatch(); break;
            case 2: sgpa(userid); break;
            case 3: goalTracker(userid); break;
            case 4: return;
        }
    } while (1);
}

void addQuiz(void) {
    FILE *fp = fopen("quiz.txt", "w");
    Quiz q;
    int n;

    if (fp == NULL) {
        printf("File error\n");
    }
}

```

```

        return;
    }

    printf("How many questions to add?: ");
    scanf("%d", &n);
    getchar(); //newline buffer eater

    for (int i = 0; i < n; i++) {
        printf("\nQuestion %d:\n", i + 1);

        printf("Enter question: ");
        fgets(q.question, sizeof(q.question), stdin);
        q.question[strcspn(q.question, "\n")] = '\0';

        printf("Option A: ");
        fgets(q.optA, sizeof(q.optA), stdin);
        q.optA[strcspn(q.optA, "\n")] = '\0';

        printf("Option B: ");
        fgets(q.optB, sizeof(q.optB), stdin);
        q.optB[strcspn(q.optB, "\n")] = '\0';

        printf("Option C: ");
        fgets(q.optC, sizeof(q.optC), stdin);
        q.optC[strcspn(q.optC, "\n")] = '\0';

        printf("Option D: ");
        fgets(q.optD, sizeof(q.optD), stdin);
        q.optD[strcspn(q.optD, "\n")] = '\0';

        do {
            printf("Correct option (A/B/C/D): ");
            scanf(" %c", &q.correct);
            q.correct = toupper(q.correct);
        } while (q.correct < 'A' || q.correct > 'D');

        getchar(); // buffer eater again

        fprintf(fp, "%s|%s|%s|%s|%s|%c\n",
                q.question, q.optA, q.optB, q.optC, q.optD, q.correct);
    }

    fclose(fp);
    printf("Questions added!\n");

```

```

    pressEnter();
}

void quiz(int userid) {
    FILE *fp = fopen("quiz.txt", "r");
    Quiz q;
    int score = 0, total = 0;
    char ans;

    if (fp == NULL) {
        printf("No quiz available\n");
        return;
    }

    while (fscanf(fp, " %199[^\n]|%99[^\n]|%99[^\n]|%99[^\n]|%99[^\n]| %c",
        q.question, q.optA, q.optB, q.optC, q.optD, &q.correct) == 6) {

        printf("\n%s\n", q.question);
        printf("A. %s\nB. %s\nC. %s\nD. %s\n", q.optA, q.optB, q.optC, q.optD);
        printf("Your answer: ");
        scanf(" %c", &ans);

        if (toupper(ans) == q.correct) {
            score++;
        }
        total++;
    }

    fclose(fp);

    printf("\nYour Score: %d/%d\n", score, total);

    Student s[MAX_STUDENTS];
    int n = loadStudents(s);
    int index = findStudent(s, n, userid);

    if (index == -1) {
        printf("Quiz Result not saved\n");
    } else {
        s[index].marks += score;
        assignGrade(&s[index]);
        saveStudents(s, n);
    }
    pressEnter();
}

```

```

}

void stopwatch(void) {
    int minutes, seconds = 0;

    system("cls");
    printf("Enter study time (minutes): ");
    scanf("%d", &minutes);

    printf("Study Timer Started for %d minutes\n", minutes);

    while (minutes >= 0) {
        printf("\rTime Left: %02d:%02d", minutes, seconds);
        Sleep(1000);

        if (seconds == 0) {
            minutes--;
            seconds = 59;
        } else {
            seconds--;
        }
    }

    printf("\n\nTime's up! Take a break.\n");
    pressEnter();
}

void sgpa(int userid) {
    int n;
    float sum = 0, totalCredit = 0;

    do {
        printf("Number of Courses: ");
        scanf("%d", &n);
    } while (n <= 0);

    for (int i = 1; i <= n; i++) {
        float credit, grade;

        printf("Credit of Course #%d: ", i);
        scanf("%f", &credit);

        printf("Grade of Course #%d: ", i);
        scanf("%f", &grade);
    }
}

```

```

        sum += credit * grade;
        totalCredit += credit;
    }

    if (totalCredit == 0) {
        printf("Invalid total credits\n");
        return;
    }

    float sgpa = sum / totalCredit;

    printf("SGPA = %.2f\n", sgpa);
    CGPA(userid, sgpa);

    FILE *fp = fopen("sgpa.txt", "a");

    if (fp != NULL) {
        fprintf(fp, "%d %.2f\n", userid, sgpa);
        fclose(fp);
    }
    pressEnter();
}

void CGPA(int userid, float newSGPA) {
    Student s[MAX_STUDENTS];
    int n = loadStudents(s);
    int index = findStudent(s, n, userid);

    if (index == -1) {
        return;
    }
    s[index].cgpa = ((s[index].cgpa * (s[index].semester - 1)) + newSGPA) /
s[index].semester;
    s[index].semester++;

    saveStudents(s, n);
}

void goalTracker(int userid) {
    Student s[MAX_STUDENTS];
    float targetCGPA;
    int n = loadStudents(s);
    int index;

```

```

system("cls");

if (n == 0) {
    printf("No records found\n");
    return;
}

index = findStudent(s, n, userid);

if (index == -1) {
    printf("Student not found\n");
    return;
}

printf("Enter your target CGPA: ");
scanf("%f", &targetCGPA);

printf("\nCurrent CGPA: %.2f\n", s[index].cgpa);
printf("Target CGPA: %.2f\n", targetCGPA);

if (s[index].cgpa >= targetCGPA) {
    printf("Congratulations!\nGoal Achieved!\n");
} else {
    printf("You need %.2f more to reach your goal.\n", targetCGPA -
s[index].cgpa);
}
pressEnter();
}

```

Limitation of Program

- **Single-word Inputs:** Names or departments with spaces (e.g., "Maliha Farzana") break standard fscanf("%s") reading.
- **Fixed Storage Limits:** The memory record processing is restricted to a maximum of 100 students at a time (Student s[100]).
- **Platform Dependency:** Uses <windows.h> (Sleep(), system("cls")), making it non-portable to Linux or macOS.
- **User Limitations:** Does not support concurrent users.

Extra Features

- Dual-portal architecture with role-based access (Faculty and Student).
- Automated attendance process and percentage tracking.
- Automated grade evaluation based on marks.
- Built-in quiz management and mark assessment modules.
- Integrated quiz and assessment systems.
- **Interactive Student Productivity Utilities:**
 - **Study Timer:** A countdown study timer with real-time screen updates to assist with time management.
 - **CGPA Calculator:** Calculates semester GPA based on credit hours & grades and automatically recalculates cumulative CGPA in the main database.
 - **Goal Tracker:** Compares the student's current CGPA against a target CGPA to calculate the necessary academic margin needed.

Conclusion

The Academic Portal System demonstrates a practical application of structured C programming concepts, including modular design, functional abstraction, and persistent file-based data handling. The software successfully fulfills core academic administrative operations while extending usability through built-in quizzes and student productivity tools. While constrained by single-threaded execution and basic flat-file storage, the application achieves functional correctness and logical efficiency. Overall, this project provides a solid foundation for future enhancements, such as SQL database integration, dynamic memory allocation, and multi-threaded, multi-user client-server support.